

OBJECT ORIENTED PROGRAMMING USING REAL TIME APPLICATIONS (DSA0135)

Abinesh M (192465050)

1. Student Analytics System

```
// STUDENT ANALYTICS SYSTEM
#include <iostream>
using namespace std;
class Student
{
private:
    string name;
    int marks;
public:
    void input() { cin >> name >> marks; }
    int getMarks() { return marks; }
    string getName() { return name; }
};
class Department
{
private:
    string deptName;
    int n;
public:
    Student s[100];
    void input()
    {
        cout<<"Department Name: ";
        cin>>deptName;
        cout<<"Number of Students: ";
        cin>>n;
        for(int i=0;i<n;i++)
            s[i].input();
    }
    void topper()
    {
        if(n==0)
        {
            cout<<"Empty Department\n";
            return;
        }
        int maxMarks=s[0].getMarks();
        string topper=s[0].getName();
        for(int i=1;i<n;i++)
        {
            if(s[i].getMarks()>maxMarks)
            {
                maxMarks=s[i].getMarks();
                topper=s[i].getName();
            }
        }
        cout<<"Topper: "<<topper<<" "<<maxMarks<<endl;
    }
    float passPercentage()
    {
        if(n==0) return 0;
        int pass=0;
        for(int i=0;i<n;i++)
            if(s[i].getMarks()>=40) pass++;
        return (pass*100.0)/n;
    }
};
int main()
{
    int d;
    cin>>d;
    Department dept[10];
```

```

for(int i=0;i<d;i++) dept[i].input();
for(int i=0;i<d;i++)
{
dept[i].topper();
cout<<"Pass % = "<<dept[i].passPercentage()<<endl;
}
return 0;
}

```

Sample Input

```

2
CSE
4
Arun 78
Bala 35
Chitra 92
Deepak 60
ECE
3
Esha 45
Farook 30
Gowri 88

```

Sample Output

```

Department Name: Number of Students: Department Name: Number of Students: Topper: Chitra 92
Pass % = 75
Topper: Gowri 88
Pass % = 66.6667

```

Note: The prompt strings ("Department Name:", "Number of Students:") print before each scanf-style read, so they appear interleaved with the input echo in a terminal. CSE department (78, 35, 92, 60) has topper Chitra with 92 marks and a 75% pass rate (3 of 4 students ≥ 40). ECE department (45, 30, 88) has topper Gowri with 88 marks and a 66.67% pass rate (2 of 3 students ≥ 40).

Time and Space Complexity

Operation	Time Complexity	Space Complexity	Explanation
Student.input() / getters	$O(1)$	$O(1)$	Reads/returns a single name and mark value
Department.input()	$O(n)$	$O(n)$	Loops once over n students; storage for n Student objects
Department.topper()	$O(n)$	$O(1)$	Single linear scan to track the running maximum
Department.passPercentage()	$O(n)$	$O(1)$	Single linear scan counting students with marks ≥ 40
main() — overall	$O(d \times n)$	$O(d \times n)$	d departments, each holding up to n students (array of size 100 per dept, 10 depts)

Here n is the number of students in a department and d is the number of departments. Each Department object statically reserves an array of 100 Student slots regardless of how many are actually used, which is why the array itself (not just the data inside it) contributes $O(100) = O(1)$ fixed overhead per department on top of the $O(n)$ live data.

2. Election Result Analysis System

```
// ELECTION RESULT ANALYSIS SYSTEM
#include<iostream>
using namespace std;
class Election
{
public:
    static int totalVotes;
    static int maxVotes;
    static int winnerIndex;
    void process(int votes[][50], int regions, int candidates)
    {
        int total[50] = {0};
        for(int j = 0; j < candidates; j++)
        {
            for(int i = 0; i < regions; i++)
            {
                total[j] += votes[i][j];
                totalVotes += votes[i][j];
            }
            if(total[j] > maxVotes)
            {
                maxVotes = total[j];
                winnerIndex = j;
            }
        }
        cout << "Total Votes = " << totalVotes << endl;
        cout << "Winner Candidate Index = " << winnerIndex + 1 << endl;
        cout << "Max Votes = " << maxVotes << endl;
    }
};
int Election::totalVotes = 0;
int Election::maxVotes = 0;
int Election::winnerIndex = 0;
int main()
{
    int regions, candidates;
    cin >> regions >> candidates;
    int votes[50][50];
    for(int i = 0; i < regions; i++)
        for(int j = 0; j < candidates; j++)
            cin >> votes[i][j];
    Election e;
    e.process(votes, regions, candidates);
    return 0;
}
```

Sample Input

```
3 3
120 95 80
110 130 60
90 100 150
```

Sample Output

```
Total Votes = 935
Winner Candidate Index = 2
Max Votes = 325
```

Note: 3 regions, 3 candidates. Column sums: Candidate 1 = 120+110+90 = 320, Candidate 2 = 95+130+100 = 325, Candidate 3 = 80+60+150 = 290. Grand total = 935. Candidate 2 wins with 325 votes.

Time and Space Complexity

Operation	Time Complexity	Space Complexity	Explanation
-----------	-----------------	------------------	-------------

Reading votes (main)	$O(r \times c)$	$O(r \times c)$	Fills a 2D array of r regions $\times$ c candidates
Election.process()	$O(r \times c)$	$O(c)$	Nested loop sums each candidate's votes across all regions; total[] array of size c (bounded by fixed size 50)
Overall	$O(r \times c)$	$O(r \times c)$	Dominated by the votes array storage and the double loop over it

Here r is the number of regions and c is the number of candidates. The votes[][] array is declared with a fixed maximum size of 50×50 , so the worst-case bound is $O(2500) = O(1)$ in terms of the maximum allowed input, but it is conventionally expressed as $O(r \times c)$ relative to the actual input size used. totalVotes, maxVotes, and winnerIndex are static members shared across all Election objects, each occupying $O(1)$ space.

3. Store Sales Management System

```
// STORE SALES MANAGEMENT SYSTEM
#include<iostream>
using namespace std;
class Product
{
public:
    string name;
    float price;
    int qty;
    void input()
    {
        cin >> name >> price >> qty;
    }
    float sales()
    {
        return price * qty;
    }
};
class Store
{
public:
    Product p[100];
    int n;
    static float overallSales;
    static string bestProduct;
    static float maxProductSales;
    void input()
    {
        cin >> n;
        for(int i = 0; i < n; i++)
            p[i].input();
    }
    float calculateStoreSales()
    {
        float total = 0;
        for(int i = 0; i < n; i++)
        {
            float s = p[i].sales();
            total += s;
            if(s > maxProductSales)
            {
                maxProductSales = s;
                bestProduct = p[i].name;
            }
        }
        overallSales += total;
        return total;
    }
};
float Store::overallSales = 0;
float Store::maxProductSales = 0;
string Store::bestProduct = "";
int main()
{
    int stores;
    cin >> stores;
    Store s[50];
    float bestStoreSales = 0;
    int bestStoreIndex = 0;
    for(int i = 0; i < stores; i++)
    {
        s[i].input();
        float total = s[i].calculateStoreSales();
        if(total > bestStoreSales)
        {
            bestStoreSales = total;
            bestStoreIndex = i;
        }
    }
    cout << "Best Store: " << bestStoreIndex + 1 << endl;
```

```

cout << "Best Selling Product: " << Store::bestProduct << endl;
cout << "Overall Sales: " << Store::overallSales << endl;
return 0;
}

```

Sample Input

```

2
3
Pen 10 100
Notebook 40 50
Bag 500 5
2
Pencil 5 200
Eraser 3 150

```

Sample Output

```

Best Store: 1
Best Selling Product: Bag
Overall Sales: 6950

```

Note: Store 1 sales: Pen $10 \times 100 = 1000$, Notebook $40 \times 50 = 2000$, Bag $500 \times 5 = 2500$, store total = 5500. Store 2 sales: Pencil $5 \times 200 = 1000$, Eraser $3 \times 150 = 450$, store total = 1450. Store 1 is the best store ($5500 > 1450$). The single highest-selling product across all stores is Bag at 2500. Overall sales across both stores = $5500 + 1450 = 6950$.

Time and Space Complexity

Operation	Time Complexity	Space Complexity	Explanation
Product.input() / sales()	$O(1)$	$O(1)$	Reads or computes values for a single product
Store.input()	$O(n)$	$O(n)$	Loops once over n products; storage for n Product objects
Store.calculateStoreSales()	$O(n)$	$O(1)$	Single linear scan accumulating total and tracking the best product
main() — overall	$O(k \times n)$	$O(k \times n)$	k stores, each with up to n products (arrays fixed at 50 stores $\times$ 100 products)

Here k is the number of stores and n is the number of products per store. overallSales, bestProduct, and maxProductSales are static members of Store, so they add only $O(1)$ shared space regardless of how many Store objects exist.